MultiChain Implementation of Adaptive Trust Management in the Internet of Vehicles

A SUMO based prototype with position and speed consistency checking

Prepared by Muhammad Khalid Khan
Date 10 October 2026
Course: Blockchain

Instructor: Dr. Shahbaz

Project outcome

The prototype runs one permissioned MultiChain network with a Traffic Authority and two roadside-unit nodes. Five vehicle identities participate in a manual trust-management test and a SUMO mobility experiment. The completed run detected two injected false-speed episodes, recorded their evidence and decisions, and transferred reward assets to reporting vehicles. A final audit found the same 46 application records on all three nodes, with every record confirmed.

Scope of the submission

The project adapts the reporting, trust and incentive workflow of Singh et al. [1]. Python implements the detector and application rules, MultiChain stores signed records and transfers assets, and SUMO supplies vehicle movement. The design uses one shared chain; it does not reproduce Ethereum smart-contract execution or regional blockchain sharding.

Verified experiment	Value
Network and version	iovlab · MultiChain Community 2.3.3
SUMO experiment	sumo-da33a5ddb2 · 100 simulated seconds
Participants	TA, RSU1, RSU2 and vehicle identities A–E
Recorded decisions	One manual session and two SUMO sessions
Reward reconciliation	66 tokens distributed; 9,934 remain with TA

Evidence convention

The console figures in this report reproduce the recorded terminal output as labeled evidence panels. They are not original desktop screenshots. No SUMO GUI screenshot was captured during the recorded run. Section 18 explains how to capture genuine interface images without changing the verified ledger. The remaining screenshot requirement should be completed before submission if the instructor requires original screen captures.

1 Paper summary

Problem

Vehicles exchange safety and traffic messages, but an authenticated sender can still provide false information. A false congestion warning or an incorrect road-safety message can affect driving decisions. Singh and colleagues investigate how vehicles can maintain a consistent reputation history, reward useful reports and restrict repeated misbehavior without relying on one central trust database. [1]

Proposed approach

The paper places blockchain-supported trust management at the roadside infrastructure. Vehicles report suspicious behavior; roadside units analyze reports and update trust through smart contracts. The Traffic Authority registers participants, while a Certificate Authority supports identity credentials. Regional Authorities manage local infrastructure and short-term identities. The proposed architecture combines a global blockchain with regional shards so that local transactions need not all burden the global ledger. [1, Sections III–V]

Platform and workflow

The implementation uses Ethereum, Geth and Solidity, with JavaScript, web3.js and Node.js for interaction. The architecture describes Proof of Work for the global chain and an authoritative validator mechanism for regional shards. A reporting session requires at least four participants and a strict majority against the suspect. The narrative reduces the suspect's trust by one and temporarily blocks it for one minute; repeated misbehavior can cause hard revocation below zero trust. A successful reporter gains one trust point and five credit points, with two extra credits for the first reporter. [1, Algorithms 1–3]

Evaluation

The reported testbed contains four PCs: one Traffic Authority and three roadside units, plus a Raspberry Pi acting as a vehicle light node. It evaluates transaction handling, including throughput and execution time. The testbed establishes implementation feasibility; the broader sharded architecture should not be treated as a fully demonstrated large-scale deployment. The paper does not report a MultiChain or SUMO implementation. [1, Sections V–VI]

Conclusion and relevance

The conclusion explicitly leaves actual misbehavior detection and local plausibility checks for future work, including position, speed, heading and beacon-related checks. It also identifies privacy, AI-assisted detection and efficient consensus as future directions. The accompanying project addresses one narrow part of that gap: a Python position–speed consistency detector driven by SUMO, with evidence and decisions recorded in MultiChain. This is a classroom adaptation, not the authors' original implementation. [1, Section VII]

[1] P. K. Singh et al., Blockchain-Based Adaptive Trust Management in Internet of Vehicles Using Smart Contract, IEEE Transactions on Intelligent Transportation Systems, accepted 2020. DOI 10.1109/TITS.2020.3004041.

2 Original architecture

The original proposal separates the vehicle plane from the roadside infrastructure and authority plane. Figure 1 redraws the proposed architecture at a conceptual level; it is not a photograph of the experimental testbed. [1, Sections III–V]

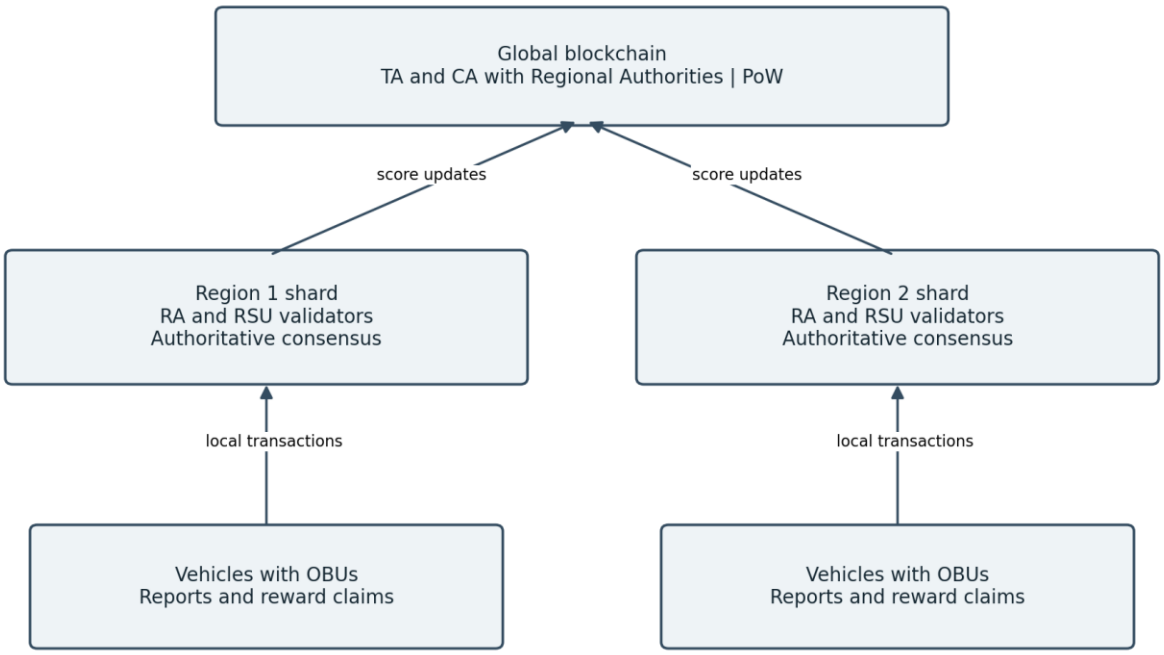


Figure 1. Redrawn conceptual architecture of the paper. Regional shards and the global chain belong to the proposal; the measured prototype is described separately.

Entity	Responsibility in the paper
Vehicle or OBU	Exchanges messages, reports suspicion and claims earned rewards.
RSU	Provides roadside computation, analyzes reports and participates in maintaining the ledger.
TA	Registers vehicles, initializes trust and authorizes roadside infrastructure.
CA	Supports registration credentials and certificates.
RA	Administers a region, its RSUs and short-term vehicle credentials.

3 Original implementation and trust logic

Platform and measured testbed

The paper uses a private Ethereum testbed. Four Ubuntu PCs run Geth: one TA and three RSUs. A Raspberry Pi runs an ARM Geth client as a vehicle light node. Solidity contracts are developed with Remix, while JavaScript requests use web3.js and Node.js over HTTP. The paper varies reporting-request batches and measures throughput and execution time. SUMO, Python and MultiChain belong to this adaptation, not to that reported software stack. [1, Section V]

Blockchain consensus and application voting

The proposed global chain uses Proof of Work. Regional shards use an authoritative mechanism in which a regional authority and RSUs validate local blocks. These are blockchain-level mechanisms. The separate application rule asking whether a majority of vehicles reports a suspect is a trust decision, not the blockchain consensus algorithm. [1, Section V]

Contract component	Purpose
Vehicle Register	Vehicle account, trust, credits and revocation state.
Personal Transaction Register	A vehicle's reports and reward status for a session.
Score and Status Register	Accumulated suspicion and the analysis result.
Session Register	Participants, first reporter and enrollment information.
Vehicle in Session Register	Vehicle reports to be considered during analysis.
Claimable Reward Register	Pending reward claims.

Rules carried into this prototype

The prototype uses at least four distinct reporters and a strict majority against a suspect. It applies one trust-point penalty per accepted suspect/session, a 60-second temporary restriction, one trust-point gain per rewarded reporter, five reward credits and a two-credit bonus for the first reporter. The initial values of trust 10 and credits 100 follow the supplied reference-contract setup. [1, Algorithms 1–3; 2]

The implementation follows the paper's narrative intent rather than translating every statement literally. In particular, the paper's analysis pseudocode places punishment inside a participant loop and adds a minute to a stored time. This prototype applies one penalty per session and sets expiry relative to the current decision time. These are explicit implementation choices.

4 Proposed MultiChain architecture

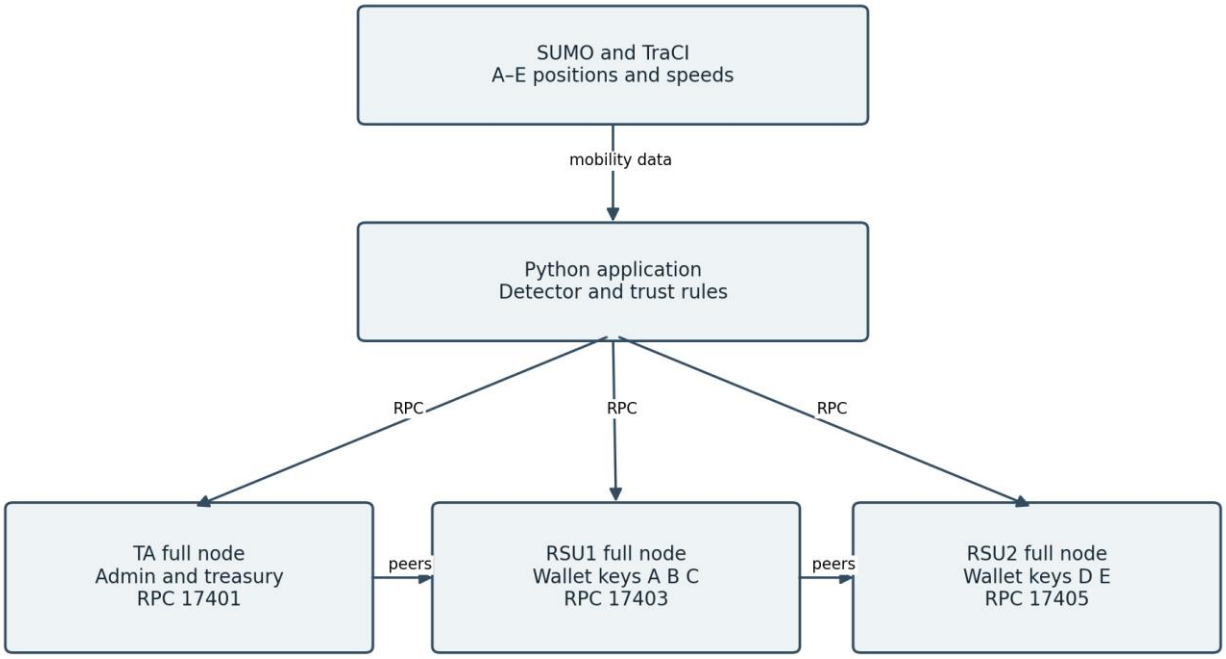


Figure 2. Implemented single-chain architecture. All three nodes run on one Windows laptop and maintain the same iovlab ledger.

TA, RSU1 and RSU2 are separate daemon processes with separate data folders, wallets and RPC ports. TA manages permissions and the reward treasury. RSUs publish decisions and selected mobility evidence. Vehicle identities are addresses: A, B and C have keys in the RSU1 wallet; D and E have keys in the RSU2 wallet. This custody arrangement makes the lab easy to demonstrate, but it does not provide independent vehicle hardware or independent control of every key.

SUMO controls road movement. A single Python process reads positions through TraCI, forms reports under the ideal sensing assumptions, and calls MultiChain through authenticated local JSON RPC. Reports are signed using publishfrom with the intended vehicle address. Region selection chooses RSU1 below x = 1,000 m and RSU2 at or beyond that point. This is a logical dispatch rule, not two separate blockchains or a simulated RSU radio network.

What is enforced where

MultiChain enforces transaction signatures, asset conservation and configured permissions. Python enforces the application's quorum, trust calculations and eligibility checks. No Solidity contract is deployed on MultiChain, and no custom transaction or stream filter was installed to independently enforce the complete trust algorithm. A privileged writer could bypass Python unless additional validation is deployed.

5 Component mapping

Original component	MultiChain equivalent	Reason and boundary
Ethereum ledger	One iovlab permissioned chain	Provides common signed transaction history; no EVM execution.
TA and part of CA role	TA node and address permissions	Registers identities; full certificate issuance is not reproduced.
Regional authority and RSU plane	RSU1 and RSU2 plus Python region selection	Demonstrates regional processing without actual shards.
Vehicle light node	A–E addresses in RSU wallets	Preserves distinct publishers, but keys are held by the lab operator.
Vehicle Register	registry plus replay of decisions and claims	Initial state stays immutable; current scores are derived from changes.
PTR and VISR	reports keyed by session and vehicle	Allows report authors and duplicate keys to be checked.
SSR and session analysis	decisions and Python vote checking	Stores accepted results and report transaction references.
Claimable rewards	requests and claims	Separates signed requests from treasury payment receipts.
Wallet incentive credits	IOVREWARD asset and credit bookkeeping	New earned rewards become transferable tokens; initial credits are not minted.
PoW and authoritative shards	Permissioned miners with mining diversity	Uses MultiChain block validation; does not reproduce Ethereum PoW economics.
Future local detection	Python detector and telemetry	Checks speed against displacement; stores auditable evidence.
Revocation logic	Python eligibility and recorded status	Temporary gate exercised; permanent threshold branch not triggered.

A stream is an append-only record collection, not a shard. Subscribing makes a node index stream contents; it does not grant permission to write. Global send permission and a restricted stream’s write permission serve different purposes. [3, 4]

6 Future direction addressed

Gap stated in the conclusion

Section VII of the paper states that local misbehavior detection was outside its implementation, including plausibility and consistency checks involving position, speed and heading. It also identifies privacy, AI-based detection and efficient roadside consensus as future directions. This project addresses only the position–speed consistency part. It does not claim to solve privacy, train an AI model, or improve consensus throughput. [1]

Detection method

At each one-second step, Python stores the previous and current position of A. It divides Euclidean displacement by the elapsed simulation time to estimate speed. The reported speed is compared with that estimate. A difference greater than 8 m/s must occur on two consecutive samples before suspicion is raised.

```
estimated_speed = math.dist(current_position, previous_position) / elapsed_time
mismatch = abs(claimed_speed - estimated_speed) > 8
streak = streak + 1 if mismatch else 0
detected = streak >= 2
```

The attack adds 30 m/s to A's reported speed during simulation seconds 15–22 and 80–87. A's actual motion is not changed. With a real speed near 15 m/s, a claimed speed near 45 m/s is inconsistent with the measured displacement. The detector does not use the attack label to decide suspicion; that label is used only to calculate evaluation metrics.

How MultiChain helps

When suspicion is accepted for reporting, an RSU writes the time, current and previous position, estimated speed, claimed speed and threshold to telemetry. Vehicle reports refer to the evidence transaction. A decision refers to its supporting report transactions. This makes the reason for a trust change traceable across nodes. The blockchain preserves what was submitted; it cannot establish that an external sensor observation is true.

Boundary of the contribution

All four reporters receive the same ideal position observation through one Python controller. Their four signatures demonstrate the reporting workflow, not four independent sensor measurements. Real deployment would need vehicle-side sensing, uncertainty handling, authenticated beacons, resistance to collusion and stronger key isolation.

7 Environment and identities

Component	Recorded version or setting
Operating system	Windows 10, build 19045.6466
MultiChain	Community 2.3.3, protocol 20013
Python	3.12.0
SUMO and TraCI	1.28.0
Python packages	requests, traci 1.28.0, sumolib 1.28.0
Working directory	C:\Users\Tesla Laptops\Downloads\multichain-windows-2.3.3
Project directory	C:\Users\Tesla Laptops\iov-lab

Node	Folder	Peer port	RPC port
TA	%USERPROFILE%\iov-lab\TA	17400	17401
RSU1	%USERPROFILE%\iov-lab\RSU1	17402	17403
RSU2	%USERPROFILE%\iov-lab\RSU2	17404	17405

Identity	Address
TA	1KvMog5Q5FuEyYvRuC6yAz6iP1DN2YDLJdgPdq
RSU1	1B9hsdERZpkBwkGGYf4DCCG6aFQRz8ifbKUWmk
RSU2	1Y14TDFJdDzjdaL5hTRVmdWvBMkzHdHXXSXXV4
A	1M4iAgVgvieWFdpRuPYFpatcbGBAVY4Q9qY1cu
B	1MrgswowcXXrCj6Mp8NYvm3ovW1apaxUMYs3ce
C	1LSsi3RYS7HcXoyPhXdzWE4y5B3AZM5pbuUZ2Z
D	1N9bGBoB6vdL2LteeVAhvVxCwWWt7cA78XQjxY
E	18UGApYRVxvobLbkPvMFAc6krrdLriwDBmpBZU

Addresses above belong to the completed chain. A newly created chain or wallet produces different addresses; reuse the commands but substitute the new addresses. Public addresses are included for auditability. RPC passwords and private keys are not included.

8 Create the chain and configure it

Open Command Prompt in the MultiChain executable directory. These creation instructions document the original setup; do not recreate the working chain before the viva.

```
cd /d "%USERPROFILE%\Downloads\multichain-windows-2.3.3"
multichain-util
py --version
sumo --version
netgenerate --version
mkdir "%USERPROFILE%\iov-lab\TA"
multichain-util create iovlab -datadir="%USERPROFILE%\iov-lab\TA"
notepad "%USERPROFILE%\iov-lab\TA\iovlab\params.dat"
```

Edit chain parameters before the first daemon start. The lab setup specified the following values. Only root-stream-open and target-block-time needed changes from the pasted generated defaults; the other listed values were retained. Runtime port overrides are separate from these chain parameters. [5]

Parameter	Value	Purpose
root-stream-open	false	Require permission for root writes.
target-block-time	2	Short target confirmation interval for the lab.
anyone-can-connect / send / receive	false	Require grants for nodes and transaction identities.
anyone-can-create / issue / mine / admin	false	Keep creation, issuance and governance permissioned.
mining-diversity	0.3	Minimum permitted-miner participation constraint.
setup-first-blocks	60	Initial setup window before normal constraints apply.
mining-requires-peers	true	Require peers, with documented setup/single-miner exceptions.
mine-empty-rounds	10	Stop idle empty-block production after the configured rounds.
initial-block-reward	0	No native-currency mining reward.
default-network-port / default-rpc-port	9231 / 9230	Generated defaults; actual ports overridden at startup.

Preserve generated chain identity and genesis fields. Do not paste a different chain's identity values. The final params.dat was not captured as a screenshot; getblockchainparams can be used to record the running chain's parameters. Mining diversity does not mean a two-thirds BFT quorum. [5]

9 Start and join the three nodes

Start the administrator

multichaind iovlab -datadir="%USERPROFILE%\iov-lab\TA" -daemon -port=17400 -rpcport=17401

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 getinfo

Initialize RSU1 and authorize its address

mkdir "%USERPROFILE%\iov-lab\RSU1"

multichaind iovlab@127.0.0.1:17400 -datadir="%USERPROFILE%\iov-lab\RSU1" -port=17402 -rpcport=17403 -daemon

The first join downloads chain parameters and prints a permission request. On the recorded chain, grant its printed address from TA:

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant 1B9hsdERZpkBwkGGYf4DCCG6aFQRz8ifbKUWmk connect,send,receive,mine

Run the same RSU1 start command again after the grant, then use getinfo.

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU1" -rpcport=17403 getinfo

Repeat for RSU2

mkdir "%USERPROFILE%\iov-lab\RSU2"

multichaind iovlab@127.0.0.1:17400 -datadir="%USERPROFILE%\iov-lab\RSU2" -port=17404 -rpcport=17405 -daemon

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant 1Y14TDFJdDzjdaL5hTRVmdWvBMkzHdHXXSXXV4 connect,send,receive,mine

Restart RSU2 with the same join command and check getinfo. Keep all three nodes running during application tests.

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 getinfo

Check a shared historical block

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 getblockhash 10

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU1" -rpcport=17403 getblockhash 10

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 getblockhash 10

All three returned the following hash. This proves agreement at height 10; the final stream audit checks the later application records as well.

000dc11999e2db1fdf2de356c1e35bd96efc0a4e6d9c13fcc37af380a44d3afa

10 Create vehicle identities and streams

Create vehicle addresses once

Call getnewaddress three times on RSU1 and label the returned addresses A, B and C. Call it twice on RSU2 and label the results D and E. getnewaddress generates a key/address pair; it does not create a vehicle or another full node.

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU1" -rpcport=17403 getnewaddress
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 getnewaddress
```

Grant send and receive to each address. Substitute A's address with B–E in the same command. All five grants succeeded on the recorded chain.

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant
1M4iAgVgvieWFdpRuPYFpatcbGBAVY4Q9qY1cu send,receive
```

Create six restricted streams

The loop below is for an interactive Windows CMD prompt. Use %%s instead of %s if converting it into a .bat file.

```
for %s in (registry reports decisions requests claims telemetry) do multichain-cli iovlab -
datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 create stream %s false
```

false restricts writing. It does not hide the contents from connected nodes. The root stream already exists, so liststreams shows seven streams in total.

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 liststreams
```

Subscribe each node

```
for %s in (registry reports decisions requests claims telemetry) do multichain-cli iovlab -
datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 subscribe %s
for %s in (registry reports decisions requests claims telemetry) do multichain-cli iovlab -
datadir="%USERPROFILE%\iov-lab\RSU1" -rpcport=17403 subscribe %s
for %s in (registry reports decisions requests claims telemetry) do multichain-cli iovlab -
datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 subscribe %s
```

A subscription creates the local indexes used to query items. On-chain data is replicated independently of subscription. The earlier comma-separated subscribe string failed because this method expects one entity name or a JSON array, not a comma-separated list. The loop fixes that syntax. [3, 4]

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 liststreams
```

Successful output showed subscribed=true and synchronized=true for all six streams. A successful subscribe response may be blank in the CLI or represented as null by the RPC interface.

11 Assign stream permissions

Writer	Allowed stream writes	Reason
TA creator	registry, claims and creator rights on all streams	Registration, treasury receipts and administrative control.
A–E	reports and requests	Vehicles submit allegations and request earned rewards.
RSU1 and RSU2	decisions and telemetry	Infrastructure records analysis and selected evidence.

For every vehicle address, execute the two grants below. The example uses A; use the address table for B–E.

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant 1M4iAgVgvieWFdpRuPYFpatcbGBAVY4Q9qY1cu reports.write
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant 1M4iAgVgvieWFdpRuPYFpatcbGBAVY4Q9qY1cu requests.write
```

Grant both RSUs decision and telemetry rights:

```
for %s in (decisions telemetry) do multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant 1B9hsdERZpkBwkGGYf4DCCG6aFQRz8ifbKUWmk %s.write
for %s in (decisions telemetry) do multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 grant 1Y14TDFJdDzjdaL5hTRVmdWvBMkzHdHXXSXXV4 %s.write
```

Verify permissions

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 listpermissions decisions.write
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 listpermissions reports.write
```

The captured decisions.write output listed RSU1, RSU2 and the TA creator address. It did not list vehicle addresses A–E. A global send grant alone does not authorize a vehicle to write a restricted decision stream. Conversely, permission to publish a decision does not mean the blockchain independently checks the application's majority calculation.

Why use publishfrom

Each RSU wallet contains several vehicle keys. publishfrom explicitly selects the identity that signs an item. The Python validator checks the resulting publishers list against the registered reporter address, rather than trusting the reporter name inside JSON. That establishes which wallet key authorized the record, although the single lab operator still controls those keys.

12 Register vehicles and issue rewards

Initial registration

Publish one registry entry for each vehicle. The example below is A's actual record. For B–E, change the stream key, vehicle name and address together. Run registration once; later score changes are appended to decisions and claims rather than overwriting registry.

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 publish registry A "{\"json\":{\"vehicle\":\"A\",\"address\":\"1M4iAgVgvieWFdpRuPYFpatcbGBAVY4Q9qY1cu\",\"initial_trust\":10,\"initial_credits\":100,\"status\":\"active\"}}"
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 liststreamitems registry
```

RSU2 returned all five records with TA as publisher and positive confirmations. initial_credits=100 is application bookkeeping. It is not a transfer of 100 IOVREWARD tokens to the vehicle.

Create the treasury asset once

```
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 issue 1KvMog5Q5FuEyYvRuC6yAz6iP1DN2YDLJdgPdq IOVREWARD 10000 1
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 subscribe IOVREWARD
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU1" -rpcport=17403 subscribe IOVREWARD
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 subscribe IOVREWARD
multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\TA" -rpcport=17401 getaddressbalances 1KvMog5Q5FuEyYvRuC6yAz6iP1DN2YDLJdgPdq
```

The quantity was 10,000 and the asset reference was 192-267-49503. The smallest unit is one whole token. Repeating issue with the same name produced error -705 because the entity already existed; it did not add another 10,000 tokens.

Install and check Python dependencies

```
py -m pip install requests traci==1.28.0 sumolib==1.28.0
py -c "import requests, traci, sumolib; print('Python packages OK')"
netgenerate --version
```

All imports succeeded and netgenerate reported SUMO 1.28.0. An unrelated invalid-distribution warning for ~inja2 appeared during pip operations, but it did not prevent the required imports or the completed simulation.

13 Python implementation and execution

File	Responsibility	How it is used
connection.py	Reads local RPC credentials, defines addresses, provides rpc and publish helpers.	Checks all nodes and registrations.
workflow.py	Runs manual-001 with signed reports, vote checks and atomic reward receipts.	Manual workflow and shared write_once helper.
simulation.py	Builds the road, runs TraCI, detects mismatches, dispatches RSUs and awards rewards.	One new run identifier per execution.
verify.py	Compares record counts, transaction-ID sets, confirmations and TA balance.	Read-only audit of the completed run.

Local configuration and RPC

connection.py reads each node's iovlab\multichain.conf from its data folder. Credentials stay on the local PC. Requests are sent to 127.0.0.1 using the relevant RPC port. Peer ports connect blockchain nodes; RPC ports receive application commands. The RPC wrapper reports node and method errors rather than printing passwords.

```
notepad "%USERPROFILE%\iov-lab\connection.py"
py "%USERPROFILE%\iov-lab\connection.py"
notepad "%USERPROFILE%\iov-lab\workflow.py"
py "%USERPROFILE%\iov-lab\workflow.py"
notepad "%USERPROFILE%\iov-lab\simulation.py"
py "%USERPROFILE%\iov-lab\simulation.py"
notepad "%USERPROFILE%\iov-lab\verify.py"
py "%USERPROFILE%\iov-lab\verify.py"
```

The four files above were created and executed during the lab. The report documents their responsibilities and key logic; the complete runnable sources remain in the project folder and should accompany the submission if source code is required.

Confirmation and duplicate checks

write_once queries an existing stream key, verifies its publisher and JSON, and only publishes if the record is absent. It then waits for matching transaction IDs with at least one confirmation on TA, RSU1 and RSU2. It rejects duplicate records for a key. This is a sequential application safeguard, not a network-wide uniqueness constraint or a concurrency-safe contract.

The manual session uses a fixed identifier and can skip existing payments. The SUMO script uses a fresh run identifier on each launch. Restarting that script creates another experiment and may issue more rewards; it is not a crash-resume mechanism.

14 SUMO scenario and application flow

Scenario setting	Value
Road	One straight 3,000 m edge, two lanes, speed limit 20 m/s
Vehicle setup	A–E; max speed 15 m/s; initial positions 200, 175, 150, 125, 100 m
Duration and seed	100 simulation seconds; 1-second steps; seed 42
Communication assumption	Ideal observations; reporting vehicles within 300 m
False-speed windows	15–22 s and 80–87 s inclusive; claimed speed adds 30 m/s
Detector	Absolute speed mismatch over 8 m/s on two consecutive samples
RSU selection	RSU1 if A's x < 1,000 m; otherwise RSU2

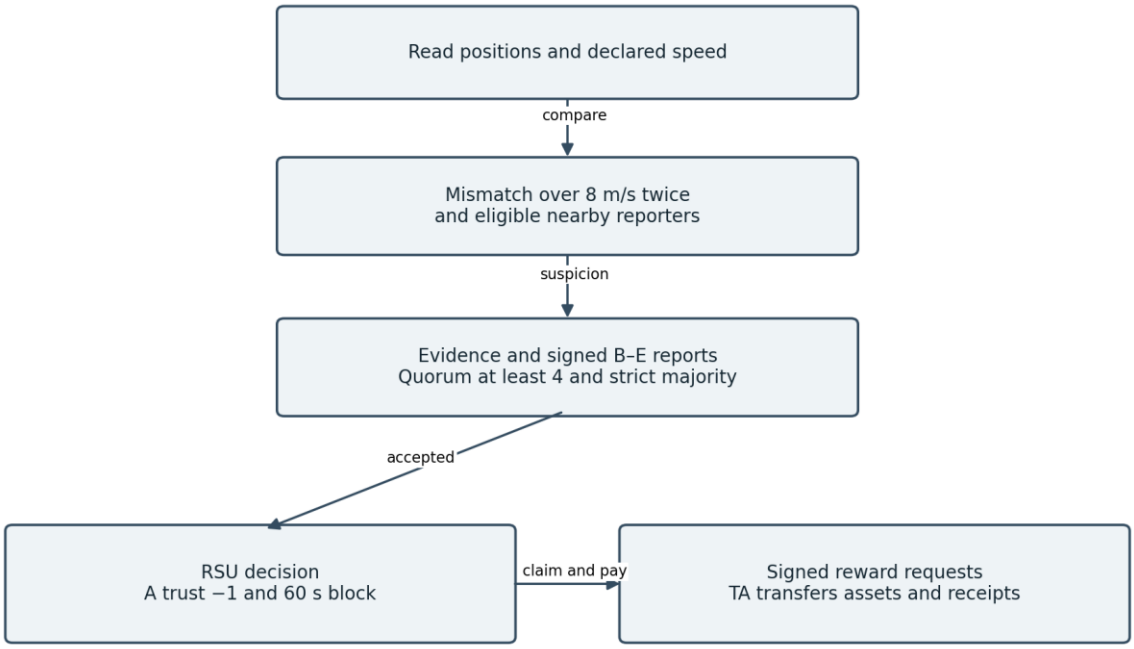


Figure 3. Implemented event flow. The eligibility and voting rules run in Python; stream permissions and asset transfers are enforced by MultiChain.

simulation.py writes road.nod.xml and road.edg.xml, calls netconvert to create road.net.xml, writes cars.rou.xml and demo.sumocfg, then starts sumo-gui through TraCI. The GUI follows A and changes its color during injected attack windows. [6, 7]

15 Decisions rewards and revocation semantics

One event from evidence to payment

At time 16, the detector had seen two consecutive speed inconsistencies. RSU1 published one telemetry item. B, C, D and E each published a report referencing that evidence. The deciding RSU read the confirmed reports, checked identities and session fields, counted four distinct reporters and accepted four positive votes. The decision reduced A's trust from 9 to 8. The same sequence occurred at time 81 through RSU2, reducing trust from 8 to 7.

Atomic reward transfer and receipt

Each reporting vehicle publishes a request. The TA sends five IOVREWARD tokens, or seven for the first reporter, and publishes the matching claims receipt in the same transaction. The receipt also records trust_delta=1 and credits_delta equal to the reward. B receives the first-reporter bonus because the demonstration submits reports sequentially in B–E order; this does not measure a real radio race. [3]

```
rpc("TA", "createrawsendfrom", TA_ADDRESS,
   {vehicle_address: {"IOVREWARD": amount}},
   [{"for": "claims", "key": session_vehicle_key,
     "data": {"json": receipt}}],
   "send")
```

Temporary blocking

After a decision, BLOCKED_UNTIL[A] is set to decision time plus 60 simulation seconds. The eligibility check requires non-negative trust and the current simulation time to be at least that expiry. A remains physically present in SUMO: blocking is an application-participation rule, not removal of a car from the road.

```
eligible = trust[vehicle] >= 0 and sim_time >= blocked_until[vehicle]
```

The first block expires at second 76, so the event at second 81 is eligible for a new decision. The second expires at 141, beyond this 100-second run. The printed block test only exercised this predicate; it did not submit a prohibited transaction and observe network rejection. No MultiChain permission was revoked.

Hard revocation and persistence boundary

The code marks negative trust as hard_revoked and excludes it through the eligibility predicate. The experiment ended at trust 7, so that branch was not exercised. Trust and credits are reconstructed from the ledger on launch; temporary expiry values are initialized for each new run and are not restored as an ongoing real-time ban. The prototype therefore does not demonstrate persistent, contract-enforced revocation.

16 Results and accounting

Stage	A trust	B trust	C D E trust	Tokens distributed
Registration	10	10	10 each	0
Manual session	9	11	11 each	22
SUMO event at 16 s	8	12	12 each	22 more
SUMO event at 81 s	7	13	13 each	22 more

Vehicle	Final trust	Final credits	Token balance
A	7	100	0
B	13	121	21
C	13	115	15
D	13	115	15
E	13	115	15

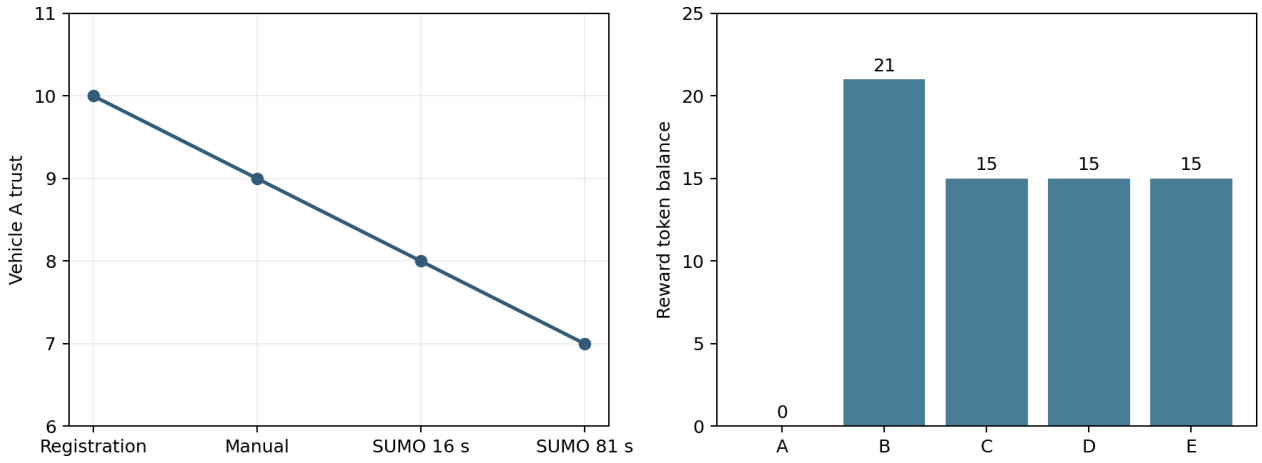


Figure 4. Trust trajectory and final token balances derived from the verified terminal output. Totals include the manual session and both SUMO sessions.

Each accepted session paid 7 + 5 + 5 + 5 = 22 tokens. Across three sessions, 66 tokens were transferred. The balance check reconciles the initial supply: 9,934 at TA plus 21 at B and 15 each at C, D and E equals 10,000. A received no reward. The initial 100 credits per vehicle are separate from this asset supply.

17 Detector evaluation and ledger audit

Actual condition	Detected mismatch	No detected mismatch
Attack sample	TP = 14	FN = 2
Normal sample	FP = 0	TN = 83

Metric	Calculation	Result
Accuracy	(14 + 83) / 99	97.98%
Precision	14 / (14 + 0)	100.00%
Recall	14 / (14 + 2)	87.50%
F1 score	28 / (28 + 0 + 2)	93.33%
Specificity	83 / (83 + 0)	100.00%

The unit of evaluation is a time sample for A, not a vehicle or a blockchain transaction. One of 100 simulation steps initializes the previous position; the remaining 99 steps are evaluated. The two eight-second attack windows contain 16 attack samples. The persistence rule misses the first sample in each window, producing two false negatives. The 14 detected samples lead to only two decision sessions because temporary blocking suppresses repeated action.

These measurements describe this single deterministic scenario. They do not establish accuracy under sensor noise, changing roads, sparse traffic or malicious collusion. No wall-clock throughput or latency comparison with Ethereum was measured.

Stream	TA	RSU1	RSU2
registry	5	5	5
reports	12	12	12
decisions	3	3	3
requests	12	12	12
claims	12	12	12
telemetry	2	2	2

verify.py compared the transaction-ID sets and expected item counts for each stream across all three nodes, then checked positive confirmations and the treasury balance. All checks passed. The 46 total entries are application records; they are not the complete chain transaction count, which also includes grants, stream creation and asset issuance.

18 Console evidence and screenshot capture

RECONSTRUCTED CONSOLE EVIDENCE
Transcribed from the user supplied Windows output

```
CHECKING THREE NODES
TA: connected | blocks=202 | vehicles A-E verified
RSU1: connected | blocks=202 | vehicles A-E verified
RSU2: connected | blocks=202 | vehicles A-E verified
TA REWARD BALANCE: IOVREWARD 10000
SETUP CHECK PASSED
```

Figure 5. Reconstructed console evidence from the successful connection check. This panel is a rendering of recorded output, not an original Windows screenshot.

RECONSTRUCTED CONSOLE EVIDENCE
Selected lines from run sumo-da33a5ddb2

```
RUN: sumo-da33a5ddb2
DETECTED t=16 | RSU1 | A trust 9->8 | soft_blocked
DETECTED t=81 | RSU2 | A trust 8->7 | soft_blocked
Vehicle | Trust | Credits | Actual reward tokens
A       |     7 |     100 | 0
B       |    13 |     121 | 21
C       |    13 |     115 | 15
D       |    13 |     115 | 15
E       |    13 |     115 | 15
Detector metrics: {'TP': 14, 'FP': 0, 'TN': 83, 'FN': 2}
Decision sessions: 2
```

Figure 6. Reconstructed console evidence from the recorded SUMO run. The selected output is preserved without representing the panel as a GUI capture.

To obtain genuine screenshots now, run connection.py and verify.py again, then use Windows Snipping Tool or Win+Shift+S. These checks do not generate new rewards. For a SUMO road image, open the existing demo.sumocfg directly as described on the next page. Label it a later mobility replay, not the original attack run.

19 Audit evidence and demonstration procedure

RECONSTRUCTED CONSOLE EVIDENCE
Transcribed from the final verify.py output

```
STREAM       TA  RSU1  RSU2  MATCH  CONFIRMED
registry      5     5     5  True   True
reports      12    12    12  True   True
decisions     3     3     3  True   True
requests     12    12    12  True   True
claims       12    12    12  True   True
telemetry     2     2     2  True   True
TA remaining tokens: 9934
PASS: All three nodes agree; all records confirmed.
```

Figure 7. Reconstructed console evidence of the final three-node audit. All counts, matching flags and the treasury balance come from the supplied verify.py output.

Read-only demonstration

py "%USERPROFILE%\iov-lab\connection.py"

py "%USERPROFILE%\iov-lab\verify.py"

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 liststreamkeyitems decisions "sumo-da33a5ddb2:16"

multichain-cli iovlab -datadir="%USERPROFILE%\iov-lab\RSU2" -rpcport=17405 liststreamkeyitems decisions "sumo-da33a5ddb2:81"

SUMO mobility replay without blockchain writes

sumo-gui -c "%USERPROFILE%\iov-lab\simulation\demo.sumocfg" --delay 100

Press Play, zoom to the vehicles, pause while they are visible and capture the window. Opening this configuration directly runs movement only: the Python attack injection, red coloring and blockchain workflow are absent. It does not change the ledger. To demonstrate those live, simulation.py must be run, which creates a new experiment and changes reward balances. [7]

Files to keep with the submission

Keep connection.py, workflow.py, simulation.py, verify.py, the generated XML configuration files, and sumo-da33a5ddb2.csv and sumo-da33a5ddb2.json from the simulation folder. Do not include wallet files, private keys or RPC passwords. Capture the actual GUI and console images if the instructor requires screenshots rather than transcribed evidence.

20 Discussion and limitations

What worked

The experiment demonstrated a full path from a mobility inconsistency to signed reports, an accepted decision and real asset rewards. Both logical RSUs participated. The same confirmed application history was visible from all three nodes, and treasury accounting reconciled exactly. These results support the feasibility of this small teaching prototype.

What remains incomplete

The adaptation does not reproduce a general EVM contract. A single trusted Python process calculates decisions and rewards, and the TA holds the treasury. Permissioned stream writers are trusted to use that process correctly. Request signatures are recorded, but the reward routine is not an independent adversarial claim-verification service. Duplicate checks are application-side and do not protect against concurrent authorized writers or arbitrary direct API calls.

A–E are five addresses controlled through two wallets on one laptop. They do not establish Sybil-resistant independent identities. The four reports share one ideal observation. There is no radio propagation, packet-loss model, GPS noise, independent evidence fusion, pseudonym rotation or location-privacy protection. The experiment considers a single targeted attacker and fixed thresholds.

The system uses one chain, so it does not evaluate sharding, regional migration of credentials, cross-chain settlement or the paper's global/local synchronization procedure. Three local processes demonstrate replication but do not establish resilience to physical host failure, partition tolerance or malicious miners. A low mining-diversity setting should not be described as a Byzantine fault-tolerance experiment.

The recorded block test was a Boolean eligibility check. Hard revocation, network-level permission revocation and persistent soft-block recovery were not demonstrated. The SUMO run is not safely resumable after an uncertain payment response. Repeating it creates a new experiment; the baseline verification script intentionally expects the original totals and will fail if more records are added.

Next improvements

A stronger version would separate vehicle keys and sensors, persist run state and block expiry, validate reward claims independently, and add filters or a suitable execution layer for critical semantic rules. It should test rejected and duplicate requests, insufficient quorum, noisy observations and repeated offenders, then evaluate multiple seeds and traffic densities. Privacy and consensus improvements require separate designs and measurements.

Conclusion

The core reporting and incentive workflow was successfully adapted to MultiChain and exercised with SUMO. The added position–speed detector addresses a specific gap identified in the paper's conclusion. Its contribution is a reproducible local experiment with an auditable evidence trail, subject to the application-trust and simulation assumptions stated above.

21 References and evidence index

[1] P. K. Singh, R. Singh, S. K. Nandi, K. Z. Ghafoor, D. B. Rawat and S. Nandi. Blockchain-Based Adaptive Trust Management in Internet of Vehicles Using Smart Contract. IEEE Transactions on Intelligent Transportation Systems, accepted manuscript, 2020. DOI: 10.1109/TITS.2020.3004041. https://doi.org/10.1109/TITS.2020.3004041

[2] kalikho. Trust-Management-ITS. Reference smart-contract repository supplied with the assignment. https://github.com/kalikho/Trust-Management-ITS

[3] MultiChain. JSON-RPC API commands. https://www.multichain.com/developers/json-rpc-api/

[4] MultiChain. Data streams. https://www.multichain.com/developers/data-streams/

[5] MultiChain. Customizing blockchain parameters. https://www.multichain.com/developers/blockchain-parameters/

[6] Eclipse SUMO. Interfacing TraCI from Python and PlainXML network definitions. https://sumo.dlr.de/docs/TraCI/Interfacing_TraCI_from_Python.html ; https://sumo.dlr.de/docs/Networks/PlainXML.html

[7] Eclipse SUMO. sumo-gui documentation. https://sumo.dlr.de/docs/sumo-gui.html

[8] MultiChain. MultiChain Web Demo, official PHP interface and configuration guide. https://github.com/MultiChain/multichain-web-demo

[9] Experimental record. Windows CMD outputs from iovlab setup, manual-001, SUMO run sumo-da33a5ddb2 and verify.py, recorded 10 October 2026. The output files are retained on the experiment PC.

Online documentation consulted 10 October 2026. Paper-specific claims refer to the supplied accepted manuscript; platform documentation describes the APIs used in the adaptation.

Evidence identifiers

Item	Identifier
Run	sumo-da33a5ddb2
Decision keys	sumo-da33a5ddb2:16 and sumo-da33a5ddb2:81
Asset issuance transaction	5fc1047f8179890387357eaa42272ad9be8bcfa7a43126dd02698b82d19a4e7c
Recorded result files	simulation\sumo-da33a5ddb2.csv and simulation\sumo-da33a5ddb2.json